

과제제안서 C211201 유영선		
과제명	LLM 기반의 동적 서사 생성을 활용한 로그라이크 게임	
팀명	직진만이 인생	
팀원	학번	이름
팀원	C311183	황준호
팀원	C211201	유영선
팀원		
팀원		

과제 개요

본 프로젝트는 Unity 기반 로그라이크 게임에 구조화된 생성형 AI를 결합하여, 플레이어의 선택과 이전 플레이 기록에 따라 다음 구간의 이야기와 사건 표현이 달라지는 동적 서사 시스템을 구현하는 것을 목표로 합니다. 플레이 흐름은 메인 메뉴, 탐색 및 선택, 미니게임, 탄막 전투, 저장과 다음 회차의 이야기 변형으로 연결합니다. 현재 구축한 JSON 기반 스토리와 환경 데이터, LLM API 연동, 저장 시스템, 전투 및 미니게임 구조를 통합하고, 학기 말에는 반복 플레이가 가능한 실행 빌드와 데이터 제작 절차를 완성합니다.

필요성 및 동기

AI 협업 개발 역량과 반복 가능한 제작 과정

생성형 AI를 실제 게임 제작 과정과 실행 시스템에 함께 적용하려면, 결과를 그대로 사용하는 방식보다 데이터 형식, 검증 기준, 저장 규칙을 먼저 설계해야 합니다. 본 프로젝트에서는 기획, 구현, 테스트, 배포 준비까지의 전 과정을 반복하며 AI가 생성한 결과를 개발자가 검토하고 승인할 수 있는 제작 흐름을 만듭니다. 이를 통해 AI 도구를 활용하더라도 게임 규칙과 품질 기준은 개발자가 통제하는 개발 방식을 실습합니다.

기존 게임과 서비스 분석

Reigns의 카드 선택 방식은 적은 입력으로도 선택 결과를 명확하게 전달하고, Undertale의 탄막 전투는 이야기 진행 중 별도의 조작 경험을 제공합니다. AI Dungeon과 같은 생성형 서사 서비스는 텍스트 확장성이 높지만, 수작업으로 구성한 3D 환경, 구조화된 게임 상태, 미니게임 및 전투 규칙과의 결합은 별도로 설계해야 합니다. 본 프로젝트는 이러한 장점을 참고하되, 자유 생성만으로 진행하지 않고 미리 정의한 JSON 구조와 게임 상태 안에서 이야기 변형이 일어나도록 범위를 제한합니다.

차별점은 플레이어 기록을 다음 회차의 이야기 변형에 반영하면서도, terrain, 등장 인물, 선택지, 해금 조건, 전투 진입 조건을 구조화된 데이터로 유지하는 데 있습니다. 따라서 반복 플레이마다 표현은 달라질 수 있지만 핵심 진행과 게임 규칙은 검증 가능한 상태로 남는다.

또한 terrain별 환경과 사건을 서로 연결하고, 이야기 속 행동이 코인 베팅, 튜토리얼 상호작용, 탄막 회피 등 서로 다른 미니게임으로 이어지도록 구성합니다. 이를 통해 생성 문장만 달라지는 게임이 아니라 환경, 선택, 규칙이 함께 변하는 플레이 경험을 구현합니다.

반복 개발과 통합 검증

개발은 기능을 작은 단위로 분리하여 구현, 플레이 테스트, 수정 순서로 반복합니다. 스토리 데이터, 환경 생성, 미니게임, 전투, 저장 기능을 독립적으로 확인한 뒤 통합하며, Git 브랜치와 커밋으로 변경 이력을 관리합니다. Codex, CLI, MCP, 플러그인과 skill은 코드 생성 자체보다 프로젝트 탐색, Unity 조작, 문서화, 반복 테스트 절차를 표준화하는 데 활용합니다.

요소 기술

공통

Unity 6.3 LTS와 C#

역할: 게임 실행 환경, 씬 전환, 상태 관리, UI, 미니게임 및 전투 로직을 구현합니다.

적용: 선택 중심 인터페이스, 3D terrain 탐색, 코인 기반 상호작용, 탄막 회피 전투를 하나의 플레이 흐름으로 통합합니다.

구조: 기능별 Controller와 데이터 클래스를 분리하여 기능 추가 시 기존 코드의 변경 범위를 줄입니다.

Git과 GitHub Desktop

역할: 팀 단위 형상 관리, 기능별 작업 분리, 변경 이력과 통합 상태 관리합니다.

적용: 개인 기능을 브랜치와 커밋 단위로 관리하고, 씬 및 prefab 충돌을 확인한 뒤 통합합니다.

Codex CLI MCP 플러그인과 skill

역할: 코드베이스 분석, Unity Editor 자동화, 반복 테스트, 에셋 및 문서 제작 보조합니다.

적용: 작업별 지침과 도구를 분리하여 개발 환경을 표준화하고, 생성 결과는 실행 및 렌더링 결과로 검증합니다.

Blender Pixilart와 Unity 에셋

역할: 2D 및 3D 게임 에셋을 제작하고 편집합니다.

적용: terrain별 시각적 특징, 캐릭터, 상호작용 오브젝트, UI 자산 구성합니다.

유영선: 게임 내부에 한국어 중심 LLM을 직접 내장하고, 플레이 중 축적되는 선택과 사건 기록을 이용해 다음 이야기의 표현을 바꾸는 동적 서사 환경을 설계합니다. 네트워크 연결이 없어도 이야기 변경 기능을 사용할 수 있도록 모델 크기, 메모리 사용량, 추론 속도를 비교하고, 게임에 포함할 수 있는 수준으로 경량화합니다. 또한 terrain마다 환경 콘셉트, 등장 인물, 사건, 선택지와 미니게임을 연결하여 이야기 변화가 실제 플레이 콘텐츠로 이어지도록 구성합니다.

황준호:

Api 기반 LLM Model

기존에 사용하던 gemini2.5flash를 비롯하여 Llama 3, Gemma 2, Mixtral, Qwen 2.5, Phi-4등 무료로 지원하는 다양한 모델을 사용하여 1학기에 있던 적은 토큰 문제를 보완합니다.

기존 기술과 구현 자산의 재사용

유영선: 현재 구현된 선택형 스토리 UI, Story JSON 구조, 플레이 기록과 세이프 데이터, 코인 베팅 및 인벤토리 구조를 계속 발전시키고 있습니다. 기존 기능을 단순히 재사용하는 데 그치지 않고 terrain별 환경 세부 구성, Encounter별 이야기와 선택 조건, 해금 요소, 미니게임 진입과 결과 반영을 추가하여 콘텐츠의 밀도와 완성도를 높이고 있습니다.

황준호: Undertale (전투 시스템): 특정 이벤트 발생 시 전환되는 탄막 회피 미니게임 인터페이스 및 조작감 벤치마킹합니다.

차별화기술

유영선: 외부 서버에만 의존하지 않고 한국어 이야기에 익숙한 LLM을 게임 내부에 내장합니다. 플레이어의 선택 기록, 현재 terrain, 등장 인물의 관계와 해금 상태를 입력으로 구성하고, 모델이 기존 이야기의 핵심 사건을 유지하면서 문장과 세부 전개를 변경하도록 학습시킨다. 생성 결과는 지정된 JSON 구조와 허용된 이야기 범위를 검사한 뒤 적용하여, 반복 플레이어의 변화와 게임 진행의 안정성을 함께 확보합니다.

황준호: 기존 Undertale 게임에서 새로운 패턴을 추가함, 날라오는 물체 자르기나 패팅 등 새로운 조작에 의한 패턴들을 추가합니다.

중점기술

유영선: terrain별 지형과 분위기는 prefab으로 구성하고, NPC, 상호작용 위치, 이야기 연결, 선택지, 해금 조건과 미니게임 설정은 JSON으로 분리합니다. Excel에서는 전체 terrain과 사건, 등장 인물, 아이템, 미니게임의 관계를 한 번에 검토하고 수정하며, 검증된 데이터를 JSON으로 변환해 Unity가 읽도록 설계합니다. 내장형 LLM은 원문 이야기와 플레이 기록을 입력받아 다음 회차에 사용할 이야기 변경안을 생성하고, 형식과 참조가 정상인 결과만 저장합니다. 이를 통해 환경 제작, 데이터 관리, 이야기 변화와 미니게임 추가를 하나의 반복 가능한 콘텐츠 제작 절차로 묶는다.

황준호: 상태 기반 전투 제어, 코루틴 기반 순차 실행 및 동적 BattleBox 변형 기술을 활용하여 탄막 회피 전투를 구현하였습니다. 또한 B1, B2, B3가 각각 독립적인 진행 순서와 조작 방식을 가질 수 있도록 전투 흐름, 실행 단계 및 개별 탄막 패턴을 분리한 모듈형 구조를 적용합니다. 이를 통해 마우스 기반 전진, 날아오는 물체 베기 및 화면 파괴 연출과 같은 새로운 전투 방식을 추가하고, 향후 패턴 확장 시 기존 시스템의 변경 범위를 최소화합니다.

상세 내용

요구 사항

유영선:

기능적 요구사항

- 한국어 중심 LLM을 게임 실행 환경에 내장하여 외부 API 연결 여부와 관계없이 이야기 변경 기능을 실행할 수 있어야 합니다.
- 플레이어의 선택 기록, 방문한 사건, 획득 아이템, 인물 관계, 해금 상태와 현재 terrain 정보를 모델 입력으로 구성해야 합니다.
- 각 terrain은 고유한 환경 prefab을 사용하고, NPC와 상호작용 오브젝트는 JSON의 식별자와 위치 정보에 따라 배치되어야 합니다.
- Excel에서 terrain, Story, Interaction, 아이템과 미니게임 데이터를 수정한 뒤 참조 오류를 검사하여 JSON 실행 데이터로 반영할 수 있어야 합니다.
- 이야기 사건에서 미니게임으로 진입하고, 성공 또는 실패 결과가 보상, 해금, 다음 선택지와 후속 이야기에 반영되어야 합니다.

비기능적 요구사항

- 내장 모델은 목표 하드웨어의 메모리 범위 안에서 실행되어야 하며, 이야기 진행을 방해하지 않는 응답 시간을 확보해야 합니다.
- 생성 결과는 인물, 장소, 이전 선택과 모순되지 않아야 하고, 요청한 JSON 필드 외의 데이터와 식별자를 변경해서는 안 됩니다.
- 새로운 terrain과 미니게임은 기존 코드를 대규모로 수정하지 않고 데이터와 모듈 추가 방식으로 연결할 수 있어야 합니다.

황준호:

기능적 요구사항:

- api통신을 통해 외부 LLM에게 기존 이야기를 전달하고, 새로운 이야기를 받아야 합니다.
- 전투 상황 발생 시 실시간 탄막 피하기 미니게임이 정상적으로 구동되어야 합니다.

비기능적 요구사항:

- 반응성: 대기 시간 내에 LLM API 통신을 완료해야 합니다.
- 일관성: LLM이 기존 단어를 적절한 단어로 대체해야 합니다.

전투가 속도감 있게 진행되어야 합니다.

시스템 설계

유영선:

전체 시스템 계층

- 콘텐츠 작성 계층: Excel에서 terrain, 등장 인물, Story, Interaction, 아이템, 해금 조건과 미니게임 목록 및 연결 관계를 관리합니다.
- 데이터 변환 및 검증 계층: Excel 항목을 JSON 구조에 대응시키고, placeId, prefabId, 이야기 경로, 해금 키의 누락과 중복을 검사합니다.
- 환경 구성 계층: terrain prefab에 고정 지형과 장식을 저장하고, JSON에 따라 NPC, 상호작용 오브젝트와 사건 위치를 배치합니다.
- 플레이 진행 계층: DecisionManager가 이야기 카드, 선택, 이동, Encounter, 미니게임 진입과 결과 반영 순서를 제어합니다.
- 내장형 LLM 계층: 플레이 기록과 현재 게임 상태를 모델 입력으로 만들고, 로컬 추론 결과의 한국어 문맥과 JSON 형식을 검증합니다.
- 저장 계층: 플레이 위치, 선택 이력, 인벤토리, 코인, 관계, 해금 상태, 모델 입력과 생성 이야기를 회차별로 저장합니다.

주요 자료구조

- 콘텐츠 데이터: terrain과 Encounter를 기준으로 인물, 이야기 경로, 해금 조건과 미니게임 연결 정보를 관리합니다.

- LLM 데이터: 플레이 기록과 현재 상태를 입력으로 구성하고, 변경 문장과 모델 버전 및 검증 결과를 저장합니다.
- 진행 데이터: 선택, 인벤토리, 코인, 관계, 해금과 미니게임 결과를 회차별로 저장합니다.

주요 알고리즘

- 콘텐츠 구성: Excel 데이터를 JSON으로 변환하고 참조 오류를 검사한 뒤 terrain prefab에 NPC와 사건을 배치합니다.
- 이야기 변형: 플레이 기록에서 필요한 정보를 추출해 내장 LLM이 이야기를 변경하고 한국어 문맥과 JSON 형식을 검증합니다.
- 결과 연결 및 복구: 미니게임 결과를 보상, 해금과 후속 이야기에 반영하며 오류가 발생하면 검증된 기본 데이터로 복구합니다.

황준호:

전체 시스템 계층:

1. 표현 계층: 사용자에게 보이는 화면과 피드백을 담당합니다.
2. 입력 계층: 현재 전투 단계에 맞는 사용자 입력을 수집합니다.
3. 전투 제어 계층: BattleScene 전체의 실행 순서와 생명주기를 관리합니다.
4. 패턴 계층: 실제 탄막과 장애물의 행동을 담당합니다.
5. 물리 및 판정 계층: BattleBox 경계 제한, 탄막 충돌, 피해 및 무적 시간, 종료 지점 도달 판정 담당합니다.
6. 데이터 계층: 피해량과 난이도 수치, 전투별 흐름 정보 저장합니다.

씬 컨트롤:

1. BattleScene은 하나의 공통 씬을 사용하되, 진입 시 현재 전투 식별값을 확인하여 B1, B2, B3 중 하나를 실행합니다.
2. 대사, 회피, 게이지 및 특수 연출의 실제 순서는 각 BattleFlow가 결정합니다.
3. 각 단계는 고정된 대기 시간만으로 연결하지 않고 대사 종료, 애니메이션 종료, 패턴 완료 및 목표 지점 도달 등의 이벤트를 기준으로 다음 단계로 전환합니다.

주요 자료구조(데이터):

1. 전투 식별 데이터(BattleType): 현재 실행할 전투를 구분합니다.
2. 전투 흐름 데이터(BattleFlowDefinition): 실행할 단계 목록, 시작 설정, 성공 조건, 실패 조건 저장합니다.
3. 패턴 설정 데이터(BattlePatternData): 패턴 이름, 지속 시간, 탄환 종류, 생성 간격, 탄환 속도, 피해량, BattleBox 크기와 위치, 플레이어 이동 모드, 종료 후 대기 시간을 저장합니다.

주요 알고리즘:

1. 전투 흐름 선택 알고리즘: BattleScene 진입 시 현재 전투 식별값을 확인 및 전투 완료시 씬을 전환합니다.
2. 탄막 패턴 선택 알고리즘:

순차 실행: 등록된 순서대로 패턴을 실행합니다.

무작위 실행: 패턴 목록에서 중복 없이 무작위로 선택합니다.

가중치 실행: 난이도와 등장 확률에 따라 패턴을 선택합니다.

조건부 실행: 체력, 경과 시간 및 이전 패턴 결과에 따라 다음 패턴을 결정합니다.

구현

운영체제: Windows 10 및 11

개발 언어: C# 및 데이터 정의용 JSON

게임 엔진과 IDE: Unity 6.3 LTS, Visual Studio 2022

데이터 도구: Excel 콘텐츠 맵, Unity Resources JSON, ScriptableObject와 prefab

AI와 자동화 도구: 유영선은 한국어 중심 LLM의 파인튜닝, 경량화 및 게임 내장형 추론 환경을 구성하고, 황준호는 외부 LLM API 통신을 이용합니다. 개발 과정에서는 Codex, CLI, MCP, 플러그인과 skill을 활용합니다.

그래픽 도구: Blender, Pixilart 및 Unity 에셋 편집 도구

협업 도구: Git, GitHub Desktop, Discord

시험평가

유영선:

- 내장 모델 시험: 인터넷 연결 없이 모델이 정상적으로 로드되고 이야기 변경 결과를 반환하는지 확인하며, 모델 크기별 메모리 사용량, 초기 로딩 시간과 평균 및 최대 추론 시간을 측정합니다.
- 이야기 품질 시험: 같은 플레이 기록을 모델별로 입력하여 한국어 자연스러움, 인물과 장소의 일관성, 핵심 사건 유지 여부, JSON 형식 준수율과 요청 외 변경 비율을 비교합니다.

- 데이터 시험: Excel과 JSON의 항목 대응을 확인하고 prefabId, placeId, Story와 Interaction 경로, 해금 키의 누락, 중복과 잘못된 참조를 검사합니다.
- 환경 및 저장 시험: terrain별 prefab과 JSON 배치 결과를 확인하고 재실행 후 위치, 선택 기록, 인벤토리, 코인, 관계, 해금과 생성 이야기가 동일하게 복원되는지 검증합니다.
- 미니게임 시험: 이야기에서 각 미니게임으로 정상 진입하는지 확인하고 성공, 실패, 취소 결과가 보상, 해금과 다음 이야기에 올바르게 반영되는지 테스트합니다.

팀원별 역할

황준호: api 통신을 통한 외부 LLM model 연결, 전투 씬의 전투 패턴 설계 및 구현을 담당합니다.

주간별/팀원별 과제 수행 계획

[illegible]